

Week 3

**Session 11:**

Date and Time Handling Concepts-

1. Java Date and Time –

**Java code:**

```
import java.io.*;
import java.util.*;
public class Solution
{
public static String findDay(int month, int day, int year)
{ Calendar cal = Calendar.getInstance();
cal.set(year, month - 1, day);
String dayName =
    cal.getDisplayName( Calendar.DAY_OF
        _WEEK, Calendar.LONG,
        Locale.ENGLISH
    );
return dayName.toUpperCase();
}
public static void main(String[] args)
{ Scanner sc = new
Scanner(System.in); int month =
sc.nextInt();
int day = sc.nextInt();
int year = sc.nextInt();
System.out.println(findDay(month, day, year));
```

VTU28055

}

}

### Sample Input

08 05 2015

### Sample Output

WEDNESDAY

## 2. Number of Days Between Two Dates –

### Java code:

```
class Solution {  
    public int daysBetweenDates (String date1,String date2)  
    { String[] d1 = date1.split("-");  
      String[] d2 = date2.split("-");  
      int y1 = Integer.parseInt(d1[0]);  
      int m1 = Integer.parseInt(d1[1]); int  
        day1 = Integer.parseInt(d1[2]);  
      int y2 = Integer.parseInt(d2[0]);  
      int m2 = Integer.parseInt(d2[1]);  
      int day2 =  
        Integer.parseInt(d2[2]);  
      return Math.abs(toDays (y1,m1,day1) - toDays (y2, m2, day2));  
    }  
    private int toDays (int year, int month, int day)  
    { int[] days = {  
        31, 28, 31, 30, 31, 30,  
        31, 31, 30, 31, 30, 31  
    };  
    int total = 0;
```

VTU28055

```
// Days from previous years  
for (int y = 1971; y < year; y++) {
```

```

        if (isLeap(y))
            total += 366;
        else
            total += 365;
    }

    // Days from previous months
    for (int m = 1; m < month; m++)
        { total += days[m - 1];
    if (m == 2 && isLeap(year)) {
        total++;
    }
    }

    // Current month's days
    total += day;
    return total;
}

private boolean isLeap(int year)
{ return (year % 400 == 0) ||
    (year % 4 == 0 && year % 100 != 0);
}
}

```

**Input:** date1 = "2019-06-29", date2 = "2019-06- 30"

**Output:** 1

## Session 12

Date and Time Handling Concepts:

### 1. Day of the Year –

#### Java code-

```
class Solution {  
    public int dayOfYear (String date) {  
        int year = Integer.parseInt(date.substring(0, 4));  
        int month = Integer.parseInt(date.substring(5, 7));  
        int day = Integer.parseInt(date.substring(8, 10));  
        int[] days = {  
            31, 28, 31, 30, 31, 30,  
            31, 31, 30, 31, 30, 31  
        };  
        int total = 0;  
        for (int i = 0; i < month - 1; i++)  
            { total += days[ i];  
            }  
        // Leap year  
        if (month > 2 && isLeapYear(year))  
            { total++;  
            }  
        total += day;  
        return total;  
    }  
}
```

```
private boolean isLeapYear(int year)
    { return (year % 400 == 0) ||
```

```
        (year % 4 == 0 && year % 100 != 0);  
    }  
}
```

**Input:** date = "2019-01- 09"

**Output:** 9

**Explanation:** Given date is the 9th day of the year in 2019.

## 2. Day of the Week –

**Java code:**

```
class Solution {  
    public String dayOfTheWeek (int day, int month, int year)  
    { String[] week = {  
        "Sunday",  
        "Monday",  
        "Tuesday",  
        "Wednesday",  
        "Thursday",  
        "Friday",  
        "Saturday"  
    };  
    int[] days = {  
        31, 28, 31, 30, 31, 30,  
        31, 31, 30, 31, 30, 31};  
    int total = 0;  
    // Count complete years from 1971
```

VTU28055

```
for (int y = 1971; y < year; y++) {
```

```

        if (isLeap(y))
            { total +=
              366;
            } else {
              total += 365;
            }
        }
    // Count complete months
    for (int m = 1; m < month; m++)
        { total += days[m - 1];
          if (m == 2 && isLeap(year))
              { total++;
            }
          }
    // Add current
    day total += day -
    1;
    // January 1, 1971 =
    Friday int index = (5 +
    total) % 7 return
    week[index];
}

private boolean isLeap(int year)
{ return year % 400 == 0 ||
  (year % 4 == 0 && year % 100 != 0);}

```

**Input:** day = 31, month = 8, year = 2019

**Output:** "Saturday"

## Session 13-

### Collection Processing & Transformation Operations-

#### 1. Java Priority Queue –

##### **Java code:**

```
import java.util.*;

class Student {
    private int id;
    private String name;
    private double cgpa;
    public Student(int id, String name, double cgpa)
    { this.id = id;
      this.name = name;
      this.cgpa = cgpa;
    }
    public int getID() {
        return id;
    } public String getName()
    { return name;
    }
    public double getCGPA ()
    { return cgpa; }
}

class Priorities {
```

```
public List<Student> getStudents(List<String> events)
{ PriorityQueue<Student> pq = new PriorityQueue<>(
```

```

        new Comparator<Student>() {
            public int compare(Student a, Student b)
            { if (a.getCGPA () != b.getCGPA ()) {
                return Double.compare(b.getCGPA (), a.getCGPA());
            }
            int nameCompare = a.getName().compareTo(b.getName());
            if (nameCompare != 0) {
                return nameCompare;
            }
            return Integer.compare(a.getID(), b.getID());
        }
    };

    for (String event : events)
    { String[] parts = event.split("
"); if
(parts[0].equals("ENTER")) {
        pq.add(new
            Student( Integer.parseInt(parts[3
]), parts[1],
            Double.parseDouble(parts[2])
        ));
    } else {
        if (!pq.isEmpty()) {
            pq.poll();
        }
    }
}

```

}

}

```
List<Student> result = new  
    ArrayList<>(); while (!pq.isEmpty()) {  
        result.add(pq.poll());  
    }  
    return result;  
}  
}
```

**Sample Output -**

Dan

Ashley

Shafaet

Maria

## 2. Java ArrayList –

### **Java code:**

```
import java.io.*;
import java.util.*;
public class
Solution{
public static void main(String[] args)
{ Scanner sc=new
Scanner(System.in); int n=sc.nextInt();
ArrayList< ArrayList<Integer>> list=new
ArrayList<>(); for(int i=0;i< n;i++){
int d= sc.nextInt();
ArrayList<Integer> row=new
ArrayList<>(); for(int j=0;j< d;j++)
row.add(sc.nextInt()); list.add(row);
}int
q= sc.nextInt();
for(int i=0;i< q;i+
+){ int
x=sc.nextInt(); int
y= sc.nextInt(); try{
System.out.println(list.get(x-1).get(y-1));
}catch(Exception e)
{ System.out.println("ERROR!");
}
}sc.close();
```

VTU28055

}

}

**Output**

74

52

37

ERROR!

ERROR!

## **Session 14 –**

### Custom Comparators:

#### 1. Largest Number –

#### **Java code:**

```
import java.util.*;

class Solution{

public String largestNumber(int[] nums)
{ String[] a=new String[ nums.length];
for(int i=0;i< nums.length;i+
+)a[i]= String.valueOf(nums[i]); Arrays.sort(a,(x,y)-
>(y+x).compareTo(x+y));
if(a[0].equals("0"))return "0";
StringBuilder s=new
StringBuilder(); for(String
x:a)s.append(x);
return s.toString();
}
}
```

**Input:** nums = [10,2]

**Output:** "210 "

VTU28055

## **Session 15-**

### **Custom Comparators & Sorting Logic:**

## 1.Sort Characters by Frequency –

### Java code:

```
import java.util.*;

class Solution{

public String frequencySort (String s)

{ Map< Character,Integer> m=new HashMap<>();

for(char c:s.toCharArray())m.put(c,m.getOrDefault(c,0)+1);

List<Character>a=new ArrayList<>( m.keySet());

a.sort((x,y)-> m.get(y)- m.get(x));

StringBuilder r=new StringBuilder();

for(char c:a)for(int i=0;i< m.get(c);i++) r.append(c);

return r.toString();

}

}
```

**Input: s** = "tree"

**Output:** "eert"

**Explanation:** 'e' appears twice while 'r' and 't' both appear once.

So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.